# Storage Engines for PageIndex Retrieval: A Workload-Grounded Benchmark of MongoDB, PostgreSQL, DuckDB, and SQLite

Junyao Dong

**Abstract.** PageIndex turns documents into large hierarchical JSON trees that an agent navigates from the root to the relevant nodes. We benchmark four storage engines (MongoDB, PostgreSQL/JSONB, DuckDB, and SQLite, ConDB's current engine) on the exact operations ConDB's retrievers issue against storage, at scales up to ten million nodes. Across navigation reads, content fetch, writes, concurrency, and storage footprint, MongoDB is competitive on every operation except one: fetching a large subtree (`get_subtree`), where its tail latency runs to seconds, more than an order of magnitude worse than the relational engines. We trace this to MongoDB's document-at-a-time read model and give an optimization plan that closes the gap on the community edition with index and schema design alone, no engine changes.

## 1 Introduction

ConDB indexes a document as a PageIndex tree: a hierarchy of nodes carrying a title, a summary, page indices, and (at the leaves) text. Retrieval is a top-down traversal: the retriever starts at the root, lists a node's children to decide where to descend, pulls a subtree for the language model to read, and fetches the content of the nodes it selects. The project question is whether a document store (MongoDB) is the right home for this data and, if so, what to optimize. This report answers both with measurements and a concrete plan.

We deliberately benchmark only the operations ConDB actually issues. Generic database operations that the retrieval path never performs (GROUP-BY aggregation, ancestor-to-root walks, page-range filters, naive substring scans) are excluded, because their results would not inform the optimization.

## 2 Workload and methodology

**Operations.** The retrievers (`beam_retriever.py`, `block_retriever.py`) reach storage through exactly four calls (Table 1). Everything measured below is one of these, plus the storage footprint, the write path (incremental index and reasoning-trace edits), and concurrency.

**Data.** Synthetic PageIndex trees in the canonical format, validated against ConDB's `DocumentTreeAdapter`, at two scales (Table 2). Content is random words; what the benchmark exercises is the tree shape and the per-node payload sizes.

**Table 1:** The storage operations ConDB retrieval issues.

| Operation | Storage call | Used for |
|---|---|---|
| Point lookup | `get_node(tree_id, node_id)` | Navigate to / read one node |
| Expand children | `get_children(tree_id, node_id)` | List children, pick where to descend |
| Get subtree | `get_subtree(tree_id, node_id, depth)` | Render the tree view, pull a block |
| Fetch content | `get_entity(tree_id, node_id)` | Read selected nodes' content |

**Table 2:** Datasets.

| Dataset | Nodes | JSON size | Depth | Fan-out |
|---|---|---|---|---|
| Medium | 70,843 | 85 MB | 5 | 6–12 |
| Large | 10,000,000 | 14.06 GB | 8 | 6–14 |

**Engines and setup.** MongoDB 7.0 (Community Edition) and PostgreSQL 16 (JSONB document column) run as servers in Docker over localhost; DuckDB and SQLite are embedded. All engines run on the same 96-core Linux host with 1 TB of RAM. Every engine stores one record per node with identical logical fields and the indexes each query needs. The tree is flattened to one row per node; `get_subtree` uses a materialized path with a byte-order range predicate, identical across engines (PostgreSQL's `path` column uses `COLLATE "C"` so its range matches the others' binary comparison). The subtree query returns node ids; adding the title and summary the tree view also renders would cost the relational engines more but not MongoDB, which materializes the whole document regardless of projection, so this choice is conservative toward MongoDB. Each read is measured over 500 sampled nodes (200 subtree roots, drawn at depth $\leq$3), fixed by seed and identical across engines; every query was checked to return the same row count on all four engines. Each concurrency level runs its client processes for five seconds. The host has ample memory, so every engine's data is cache-resident and reads are not disk-bound.

**Fairness caveats.** SQLite and DuckDB run in-process, so a query is a function call; MongoDB and PostgreSQL pay per-call client–server overhead (a localhost round trip plus protocol handling) that the embedded engines never incur. The fair single-call comparison is therefore MongoDB versus PostgreSQL; absolute microseconds across the embedded/server line are not comparable, and the concurrency section (independent processes, no GIL) shows real server throughput. MongoDB's `storageSize` is post-compression (WiredTiger snappy); the others are uncompressed on-disk files, so MongoDB's uncompressed logical size is reported separately, and the random-word content likely compresses better than real prose, flattering the compressed figure. All measurements are single-host with one tree per database; multi-tenant selectivity is not exercised.

# 3 Results

## 3.1 Storage and ingest

DuckDB and MongoDB are the most compact (Table 3), DuckDB through columnar encoding and MongoDB through compression; even uncompressed, MongoDB's 15.21 GB of BSON is no larger than the relational engines' raw tables. PostgreSQL and SQLite are several times larger on disk.

**Table 3:** Storage and ingest (large dataset). On-disk totals include the indexes. MongoDB holds 15.21 GB of logical BSON in 5.27 GB of data on disk (WiredTiger snappy).

| Engine | On-disk | Index | Uncompr. | Ingest (n/s) | Build |
|---|---|---|---|---|---|
| DuckDB | **5.02 GB** | 0 | – | **348,831** | **10 s** |
| MongoDB | 5.81 GB | 536 MB | 15.21 GB | 41,387 | 69 s |
| PostgreSQL (JSONB) | 17.71 GB | 1.19 GB | – | 62,816 | 52 s |
| SQLite | 19.51 GB | 980 MB | – | 52,880 | 38 s |

## 3.2 Retrieval operations

The navigation reads (point lookup, children, content fetch) are sub-millisecond on every engine but DuckDB, which is not built for OLTP point reads; here MongoDB keeps pace with the relational engines (Table 4). Only `get_subtree` costs at scale, and the gap is widest at the tail: a large subtree returns tens of thousands of documents, and MongoDB's P95 runs to seconds while the others stay under ∼150 ms (Figure 1). The materialized-path range also beats a recursive `parent_id` traversal, so ConDB should keep it.

**Table 4:** Retrieval read latency (ms, lower is better). Best per row in bold.

| Operation | MongoDB | PostgreSQL | DuckDB | SQLite |
|---|---|---|---|---|
| *Medium, P50* | | | | |
| Point lookup (`get_node`) | 0.238 | 0.081 | 2.705 | **0.008** |
| Expand children (`get_children`) | 0.251 | 0.090 | 0.856 | **0.017** |
| Get subtree (`get_subtree`) | 0.498 | 0.210 | 2.700 | **0.091** |
| Fetch content (`get_entity`) | 0.228 | 0.071 | 2.750 | **0.007** |
| *Large, P50 / P95* | | | | |
| Point lookup | 0.270 / 0.32 | 0.087 / 0.09 | 5.59 / 8.97 | **0.012** / **0.014** |
| Expand children | 0.282 / 0.32 | 0.099 / 0.14 | 4.28 / 5.05 | **0.023** / **0.032** |
| Get subtree (∼36k rows) | 30.9 / 2478 | 10.7 / 123 | **9.9** / **47** | 11.8 / 135 |
| Fetch content | 0.249 / 0.32 | 0.081 / 0.09 | 5.21 / 8.35 | **0.012** / **0.013** |

## 3.3 Write path

On the write path (Table 5), PostgreSQL posts the lower update latency of the two servers, but each `jsonb_set` versions the row and leaves a dead tuple for a later `VACUUM`. WiredTiger is MVCC too, yet it reclaims obsolete document versions automatically, so an update-heavy workload carries no growing bloat; its measured on-disk delta is small but noisy, since WiredTiger reports storage in checkpoint-sized chunks. DuckDB is unsuitable for OLTP writes.

## 3.4 Concurrency

Single-call latency understates the server engines. Driving point lookups from a growing pool of independent OS processes (no GIL bottleneck), both server engines gain throughput with added clients, and the gap to the embedded engines narrows far below what the single-call numbers suggest (Figure 2). MongoDB rises steadily across the whole range; PostgreSQL peaks near 16 clients;

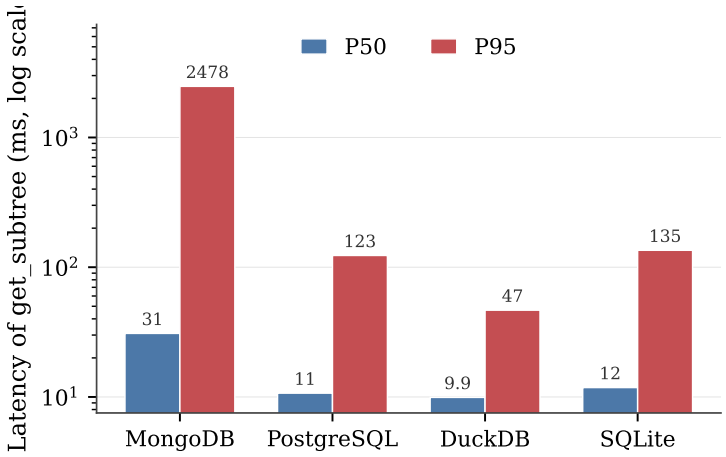


**Figure 1:** `get_subtree` latency on the large dataset (log scale). The medians are within a small factor; MongoDB's P95 is the outlier at 2.5 s.

**Table 5:** Write path (medium): 5,000 field updates, 2,000 incremental inserts. "Bloat" is on-disk growth from the updates.

| Engine | Upd. P50 | Upd. ops/s | Insert ops/s | Bloat |
|---|---|---|---|---|
| MongoDB | 0.242 | 4,050 | 5,586 | **0 MB** |
| PostgreSQL (JSONB) | 0.123 | 7,856 | 8,732 | 7 MB |
| DuckDB | 2.966 | 332 | 450 | 7 MB |
| SQLite | **0.017** | **40,442** | **21,815** | **0 MB** |

SQLite is fastest in absolute terms but peaks earliest, within the first few clients, then degrades as the processes contend on its single file. Repeated runs move the peak positions and heights somewhat; the shapes hold.

## 4 Why MongoDB trails on `get_subtree`

The cause is structural, not a configuration mistake. MongoDB's unit of read is the whole document. A secondary index (here on `path`) is a separate B-tree holding only the indexed field and a record id; to return any other field, the engine must *fetch* the full BSON document and apply the projection to it. A point lookup is one such fetch; a large subtree is tens of thousands, the biggest in the workload far more. The data is cache-resident (and pages in the WiredTiger cache are stored uncompressed), so the cost is neither disk I/O nor decompression: the time goes to per-document work, locating each record in the B-tree and materializing the full document, `text` payload included, only to project out a node id, then batching tens of thousands of results through the cursor. Multiplied by subtree size, that per-document overhead is the multi-second tail. The other operations touch few documents, so they stay fast.

The tail is super-linear as well. All engines answer the same 200 subtree queries, so the spread of their latency distributions is comparable. PostgreSQL and SQLite sit at a P95/P50 ratio of $\sim$11$\times$, tracking the spread of subtree sizes; MongoDB sits at 80$\times$, so on the largest subtrees its cost per document also rises, compounding the document-at-a-time model.

The row and columnar engines do less work per node. A row store visits each matching row but

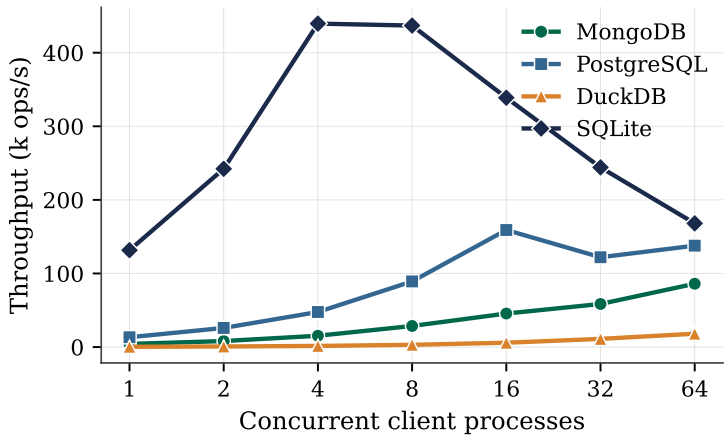


**Figure 2:** Point-lookup throughput versus concurrent client processes (medium).

materializes only the requested column; the wide `text` sits unread in the row (PostgreSQL does not even decompress the JSONB datum, since `node_id` is a separate column). A column store reads only the requested column in vectorized batches. Both could go further still with a covering index that answers the range from the index alone (optimization 1 below). All of these do far less per-row work than a per-document fetch.

This is the price of the document model, and it is also why MongoDB wins elsewhere: storing all of a node's fields together makes "fetch one whole entity" fast (point and children reads), in-place field updates cheap (the write path), the layout shard-friendly (concurrency), and the on-disk form compact (storage). MongoDB is optimized for a different access pattern, and the PageIndex workload plays to it on every operation but one.

## 5 Optimization plan

The target is `get_subtree`; nothing else needs work. All of the following is index and schema design on MongoDB Community Edition, with no source changes.

1. **Covering index** on `path` plus the fields the tree view renders (`node_id`, `title`, `summary`), with the projection limited to those fields and `_id:0`. `get_subtree` then runs index-only and never fetches each node's large `text` body, which is the source of the tail. Highest leverage, pure configuration; the cost is a larger index.

2. **Split the large `text` field** into a separate collection (structure and summary in the node, content keyed by node id). Smaller hot documents make every fetch and the working set cheaper.

3. **Nested-document subtree model.** Store bounded subtrees as single nested documents, so a subtree read is a handful of fetches instead of 36,000. A full-text subtree would blow past MongoDB's 16 MB document limit several times over, so this works in combination with the text split of item 2: structure-and-summary buckets of a few levels each stay well under the limit. Largest potential win; the trade-off is that per-node updates become rewrites of the enclosing document.

4. **Prefer the materialized-path range** form of `get_subtree` over recursive `parent_id` traversal.

Point, children, and content reads, the write path, concurrency, and storage are already MongoDB's strengths and need no work. Ad-hoc analytics over the whole corpus (which PageIndex retrieval does not perform) would require a columnar side store rather than a change to MongoDB.

## 6 Conclusion

For the PageIndex retrieval workload, MongoDB is competitive on point and children reads, content fetch, and concurrency, accumulates no dead-tuple bloat under updates (no vacuum debt), and has a compact footprint. Its one weakness is the tail latency of large `get_subtree` calls, a direct consequence of reading whole documents one at a time. That gap is closable on the community edition with covering indexes, a split content collection, and a nested subtree model, while keeping MongoDB's write, concurrency, and footprint advantages. The optimization effort should target `get_subtree` and nothing else.